

Experiment Report

Course: Computer Programming

Experiment 4: Object-Oriented Programming

Class:

Name:

Student ID:

Instructor:

Date: 2026.6.25

Experiment 4: Object-Oriented Programming

I. Objectives and Content

This experiment aims to practice object-oriented programming and classes, which is Chapter 7 of the textbook. The detailed goals of this experiment are as follows.

- **OOP Paradigm:** Understand the transition from procedural programming to object-oriented programming using classes and objects.
- **Class Anatomy:** Define data members (attributes) and member functions (behaviors) in a class.
- **Object Lifecycle:** Understand default constructors, parameterized constructors, and destructors.
- **Encapsulation:** Use public and private access specifiers to protect data and maintain valid object states.
- **File Modularity:** Separate class interfaces, implementations, and application code using .h and .cpp files.

Complete the following 4 programming tasks. Each task contains several sub-tasks. Ensure all code includes proper documentation and follows the specific technical requirements for each problem.

II. Preparation

This preparation aims to review the basic knowledge required for Experiment 4: Object-Oriented Programming. The main preparation work is as follows.

- **OOP Paradigm:** Preview the basic concepts of classes and objects, and understand how to organize programs using object-oriented thinking.
- **Class Anatomy:** Review how to define private data members and public member functions in a class.
- **Object Lifecycle:** Preview the use of default constructors, parameterized constructors, and destructors.
- **Encapsulation:** Understand how setter and getter functions protect private data and maintain valid object states.
- **File Modularity:** Review how to separate class declarations, function definitions, and application code using .h and .cpp files.
- **Tools:** Prepare Visual Studio Code and a C++ compiler for editing, compiling, and running the programs.

III. Project Results

Task 1: Rectangle Class (Basics, Constructors, and Arrays)

1. **Description:** Describe the class design and program logic in words or with a flow diagram. Do not paste the source code here.

- 1) Create a Rectangle class with private data members width and height. And add a default constructor that sets both dimensions to 1.0 and a parameterized constructor that initializes both dimensions.

```
class Rectangle
{
    private:
        double width;
        double height;

    public:
        Rectangle() : width(1.0), height(1.0) {}

        Rectangle(double w, double h) : width(1.0), height(1.0)
        {
            setWidth(w);
            setHeight(h);
        }
}
```

- 2) Add setWidth() and setHeight() functions. Only positive values may update the data members.

```
void setWidth(double w)
{
    if (w > 0)
    {
        width = w;
    }
    else
    {
        cout << "Warning: Invalid width value " << w << " rejected. Width
remains " << width << "." << endl;
    }
}

void setHeight(double h)
{
    if (h > 0)
    {
        height = h;
    }
    else
    {
        cout << "Warning: Invalid height value " << h << " rejected. Height
remains " << height << "." << endl;
    }
}
```

- 3) Define a public member function calculateArea() that returns the area.

```
double calculateArea() const
{
    return width * height;
}
```

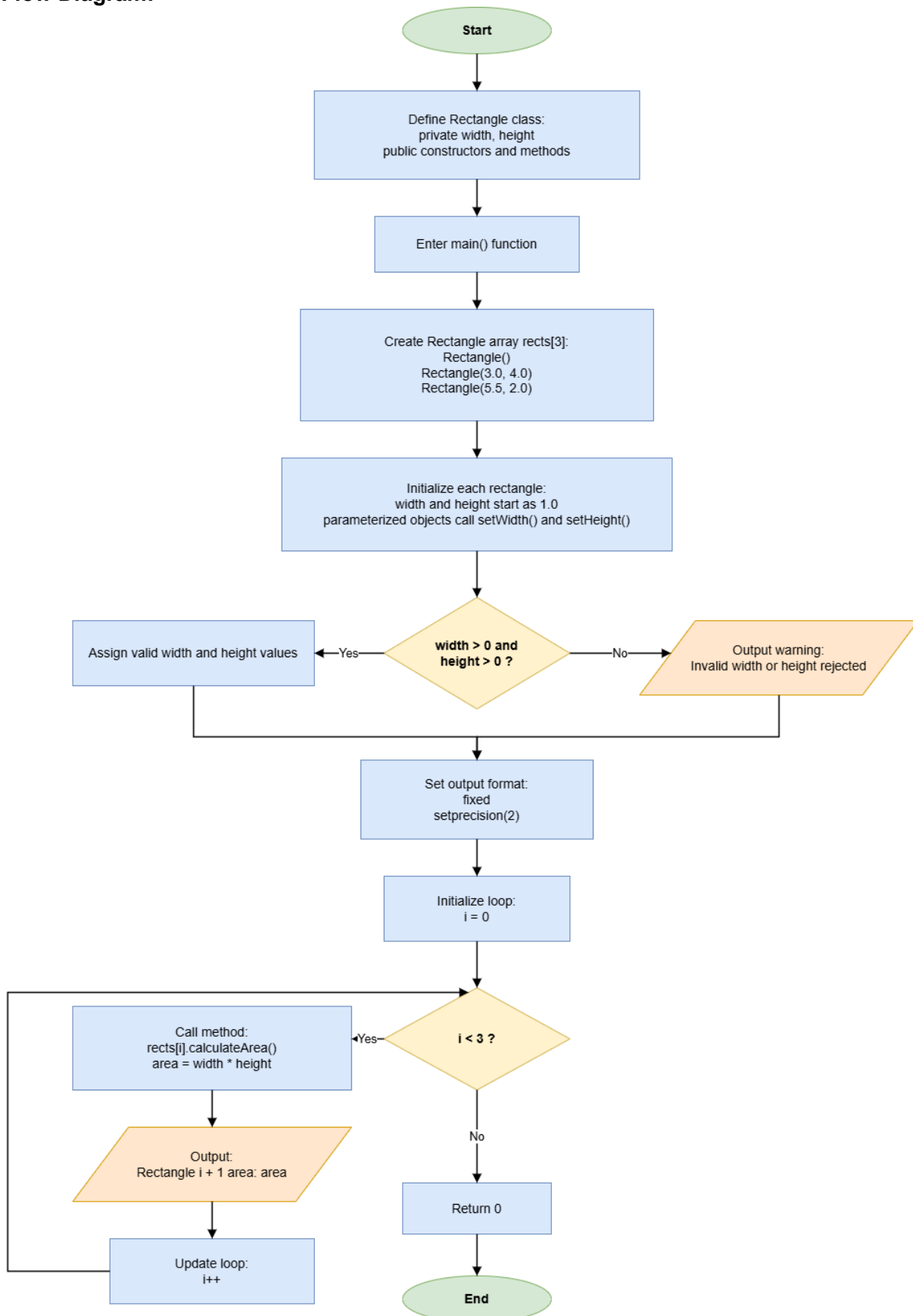
- 4) In main(), create an array of 3 Rectangle objects with different dimensions.

```
Rectangle rects[3] =
{
    Rectangle(),
    Rectangle(3.0, 4.0),
    Rectangle(5.5, 2.0)
};
```

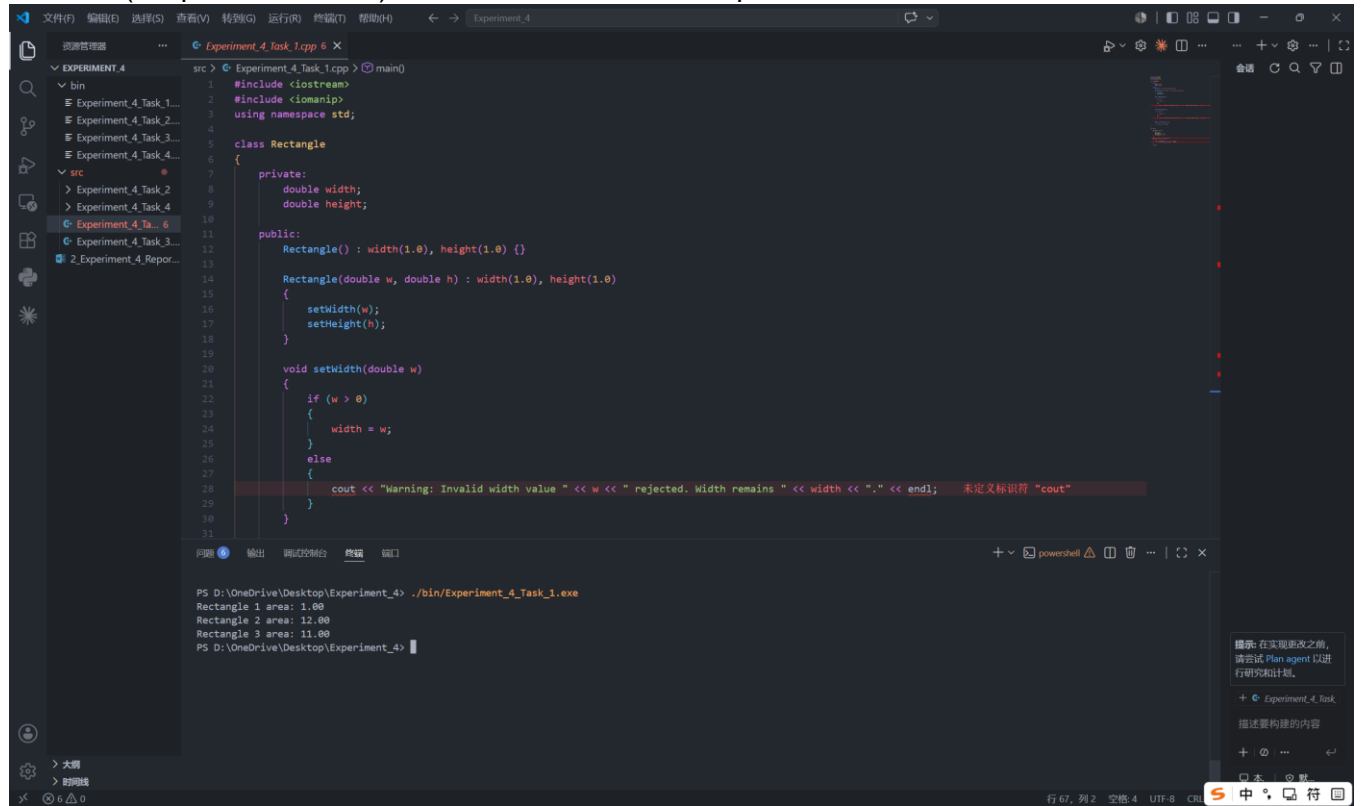
- 5) Display their areas using a loop.

```
cout << fixed << setprecision(2);
for (int i = 0; i < 3; i++)
{
    cout << "Rectangle " << i + 1 << " area: "
        << rects[i].calculateArea() << endl;
}
```

2. Flow Diagram:



3. Results (Output Screenshot): The screenshot of the output of the code.



The screenshot displays a C++ development environment with the following components:

- Source Code (Experiment_4_Task_1.cpp):**

```
src> C:\Experiment_4_Task_1\src> main()
1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5  class Rectangle
6  {
7  private:
8      double width;
9      double height;
10
11 public:
12     Rectangle() : width(1.0), height(1.0) {}
13
14     Rectangle(double w, double h) : width(1.0), height(1.0)
15     {
16         setWidth(w);
17         setHeight(h);
18     }
19
20     void setWidth(double w)
21     {
22         if (w > 0)
23         {
24             width = w;
25         }
26         else
27         {
28             cout << "Warning: Invalid width value " << w << " rejected. Width remains " << width << "." << endl; 未定义标识符 "cout"
29         }
30     }
31 }
```
- Terminal Output:**

```
PS D:\OneDrive\Desktop\Experiment_4> ./bin/Experiment_4_Task_1.exe
Rectangle 1 area: 1.00
Rectangle 2 area: 12.00
Rectangle 3 area: 11.00
PS D:\OneDrive\Desktop\Experiment_4> |
```

Task 2: Time Class (Logic & Multi-file Structure)

1. **Description:** Describe the class design and program logic in words or with a flow diagram. Do not paste the source code here.

- 1) Design a Time class with private data member hour, minute, and second using int in the header file Time.h. And declare the functions print() and tick().

```
#ifndef TIME_H
#define TIME_H

class Time
{
    private:
        int hour;
        int minute;
        int second;

    public:
        Time(int h, int m, int s);
        void print() const;
        void tick();
};

#endif
```

- 2) Define the functions in Time.cpp.

Define a print() function that displays the time in HH:MM:SS format.

Define a tick() function that increments the time by one second. When second reaches 60, it becomes 0 and minute increases by 1. When minute reaches 60, it becomes 0 and hour increases by 1. When hour reaches 24, it becomes 0

```
#include "Time.h"
#include <iostream>
#include <iomanip>
using namespace std;

Time::Time(int h, int m, int s) : hour(h), minute(m), second(s) {}

void Time::print() const
{
    cout << setw(2) << setfill('0') << hour << ":" << setw(2) << setfill('0') << minute
    << ":" << setw(2) << setfill('0') << second;
}

void Time::tick()
{
    second++;
    if (second >= 60)
    {
        second = 0;
        minute++;
    }
}
```

```

        if (minute >= 60)
        {
            minute = 0;
            hour++;
            if (hour >= 24)
            {
                hour = 0;
            }
        }
    }
}

```

3) Ask the user to input the initial hour, minute, and second.

```

int h, m, s;

cout << "Enter hour minute second: ";
cin >> h >> m >> s;

```

4) Print the initial time.

```

cout << "Initial time: ";
t.print();
cout << endl;

```

5) Loop tick() 100 times and print the final time.

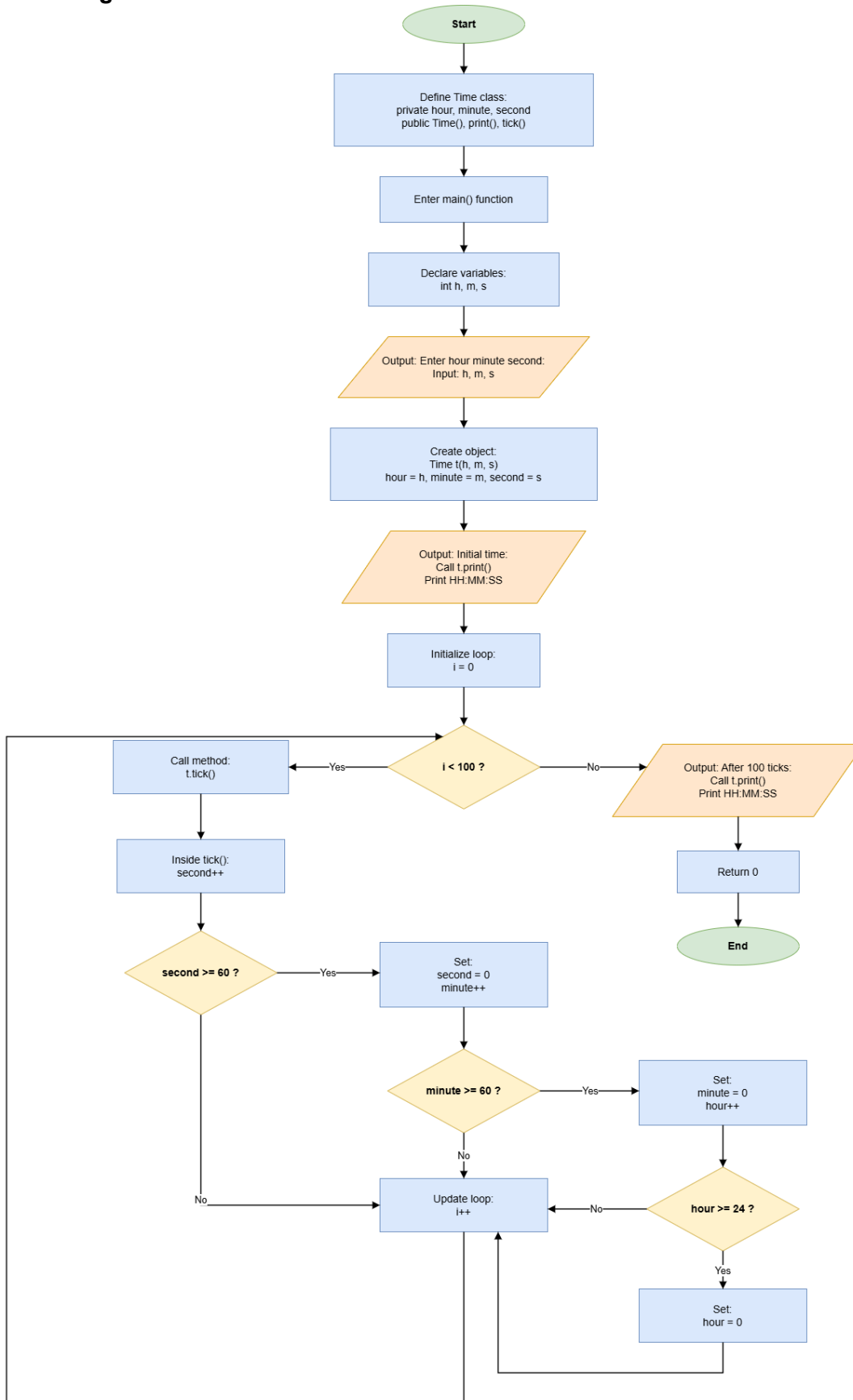
```

for (int i = 0; i < 100; i++)
{
    t.tick();
}

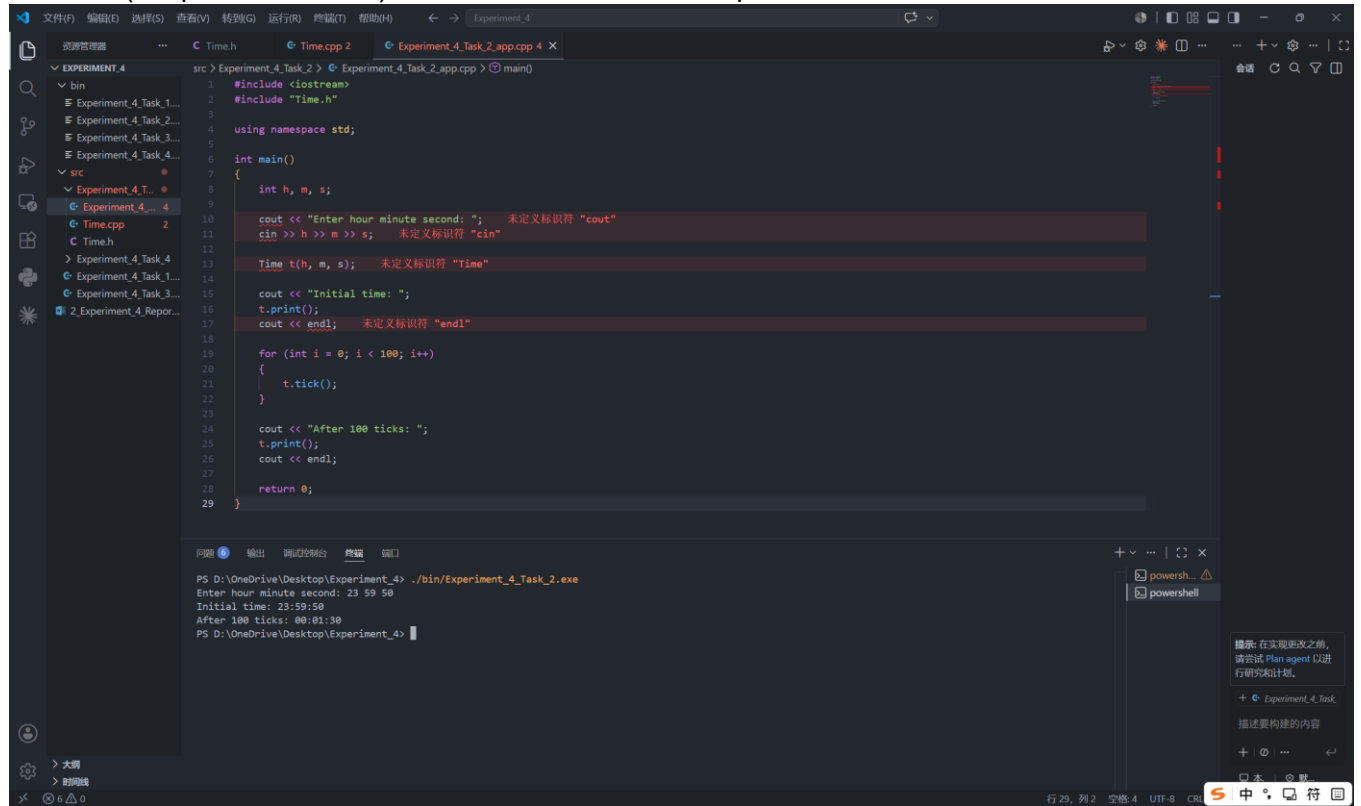
cout << "After 100 ticks: ";
t.print();
cout << endl;

```

2. Flow Diagram:



3. Results (Output Screenshot): The screenshot of the output of the code.



The screenshot displays the Visual Studio Code editor with a C++ project named 'Experiment_4'. The file explorer on the left shows the project structure, including 'src' and 'bin' directories. The main editor window shows the source code for 'Experiment_4_Task_2_app.cpp'. The code includes `<iostream>` and `"Time.h"`, uses the `std` namespace, and defines a `main` function. Inside `main`, it declares variables `h`, `m`, and `s`, prompts the user to enter hour, minute, and second, and then creates a `Time` object `t` with these values. It prints the initial time and then enters a loop that calls `t.tick()` 100 times. After the loop, it prints the time again and returns 0.

```
1 #include <iostream>
2 #include "Time.h"
3
4 using namespace std;
5
6 int main()
7 {
8     int h, m, s;
9
10    cout << "Enter hour minute second: "; 未定义标识符 "cout"
11    cin >> h >> m >> s; 未定义标识符 "cin"
12
13    Time t(h, m, s); 未定义标识符 "Time"
14
15    cout << "Initial time: ";
16    t.print();
17    cout << endl; 未定义标识符 "endl"
18
19    for (int i = 0; i < 100; i++)
20    {
21        t.tick();
22    }
23
24    cout << "After 100 ticks: ";
25    t.print();
26    cout << endl;
27
28    return 0;
29 }
```

The output window at the bottom shows the execution results in a PowerShell terminal. The output matches the code's logic, showing the initial time and the time after 100 ticks.

```
PS D:\OneDrive\Desktop\Experiment_4> ./bin/Experiment_4_Task_2.exe
Enter hour minute second: 23 59 50
Initial time: 23:59:50
After 100 ticks: 00:01:30
PS D:\OneDrive\Desktop\Experiment_4>
```

On the right side of the interface, there is a sidebar with a message in Chinese: '提示: 在实施更改之前, 请尝试 Plan agent 以进行研究计划。' (Tip: Before implementing changes, please try Plan agent for research planning.) Below this message are buttons for '+ Experiment_4_Task', '+ ...', and '+ ...'.

Task 3: Student Class (Encapsulation & Data Handling)

1. **Description:** Describe the class design and program logic in words or with a flow diagram. Do not paste the source code here.

- 1) Create a Student class with private data members name, id, and grade. Declare the functions setName(), setID(), setGrade(). And initialize the grade as 0.0.

```
class Student
{
    private:
        string name;
        string id;
        double grade;

    public:
        Student() : grade(0.0) {}

        Student(string n, string i, double g) : name(n), id(i), grade(g)
        {
            setName(n);
            setGrade(g);
            setId(i);
        }
}
```

- 2) Add getter and setter functions for all attributes.

```
void setName(string n)
{
    if (!n.empty())
    {
        name = n;
    }
    else
    {
        cout << "Warning: Invalid name value \"" << n << "\" rejected. Name
remains \"" << name << "\"." << endl;
    }
}

void setGrade(double g)
{
    if (g >= 0.0 && g <= 100.0)
    {
        grade = g;
    }
    else
    {
        cout << "Warning: Invalid grade value \"" << g << "\" rejected. Grade
remains \"" << grade << "\"." << endl;
    }
}
```

```

    }

    void setId(string i)
    {
        if (!i.empty())
        {
            id = i;
        }
        else
        {
            cout << "Warning: Invalid ID value \"" << i << "\" rejected. ID remains \"" << id << "\"." << endl;
        }
    }

    string getName() const
    {
        return name;
    }

    string getId() const
    {
        return id;
    }

    double getGrade() const
    {
        return grade;
    }

```

- 3) Define a public member function displayInfo() to print all student details.

```

    void displayInfo() const
    {
        cout << "Name: " << name << ", ID: " << id << ", Grade: " << fixed <<
        setprecision(2) << grade << endl;
    }

```

- 4) In main(), create an array of at least 3 Student objects.

```

Student students[3] =
{
    Student("Alice", "2024001", 95),
    Student("Bob", "2024002", 82.5),
    Student("Cindy", "2024003", 76)
};

```

- 5) Display their information using a loop.

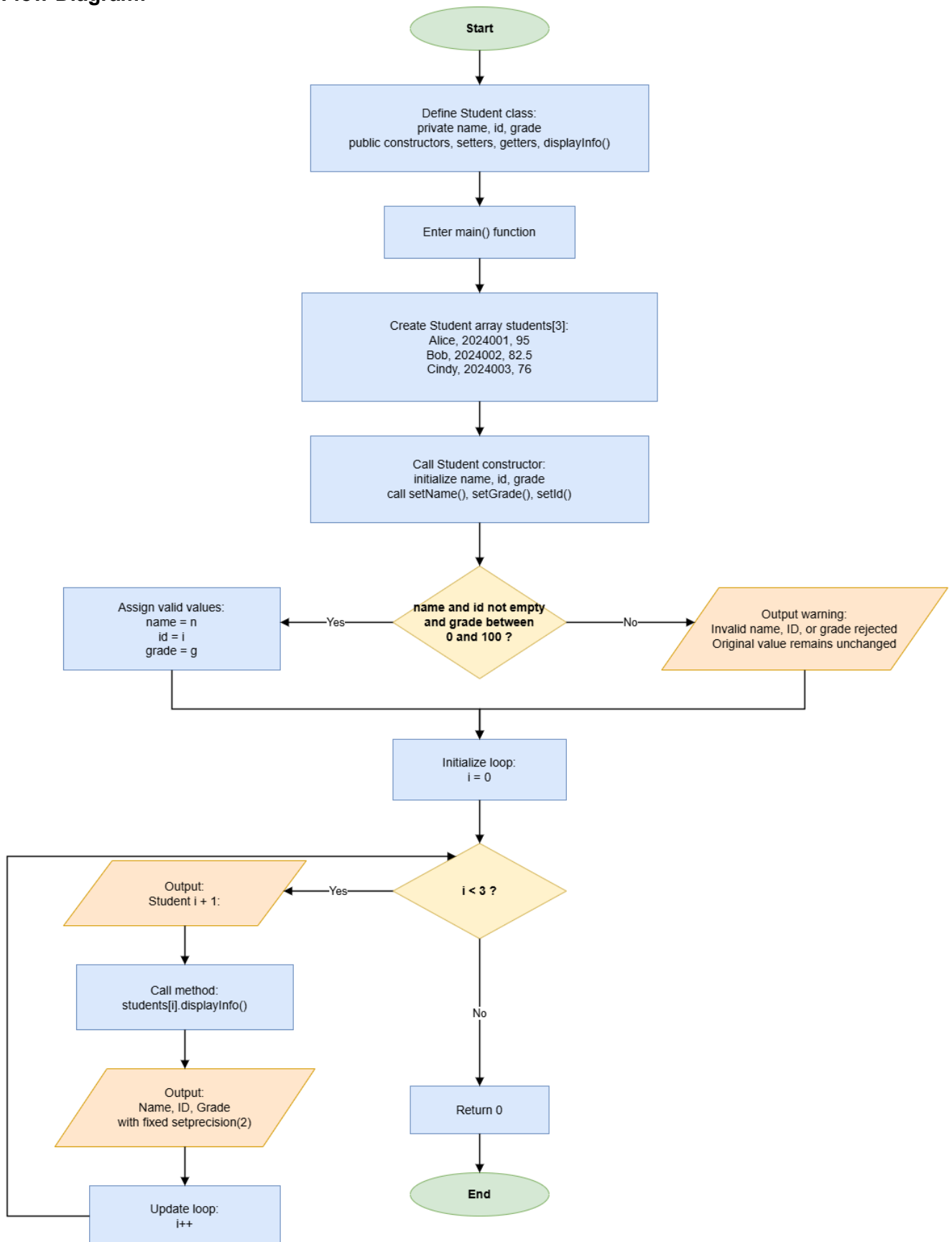
```

for (int i = 0; i < 3; i++)
{
    cout << "Student " << i + 1 << ": ";
    students[i].displayInfo();
}

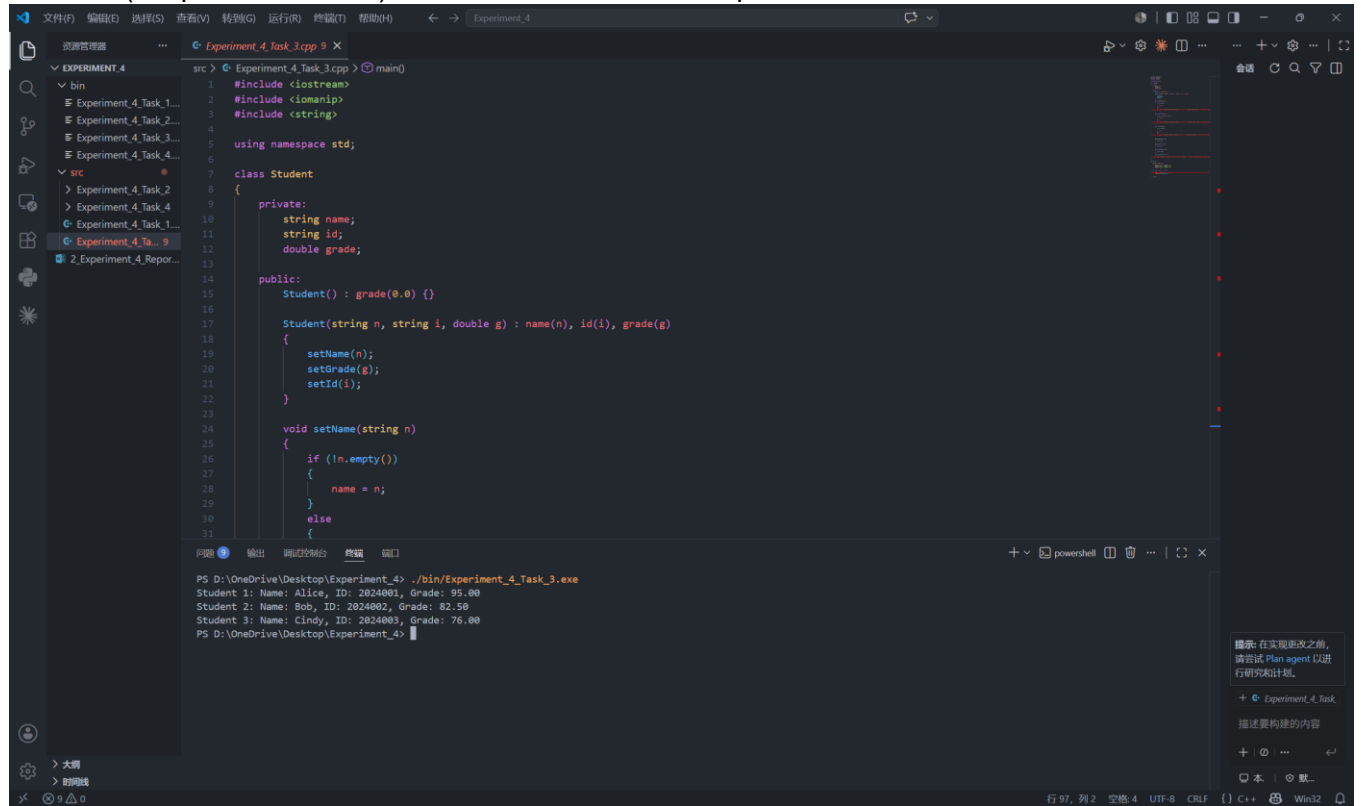
```

}

2. Flow Diagram:



3. Results (Output Screenshot): The screenshot of the output of the code.



The screenshot displays the Visual Studio Code interface with a C++ project named 'Experiment_4'. The file explorer on the left shows the project structure, including a 'src' directory containing 'Experiment_4_Task_3.cpp'. The main editor window shows the code for 'Experiment_4_Task_3.cpp', which defines a 'Student' class with private attributes 'name', 'id', and 'grade', and public methods for initialization and setting values. The 'main' function creates three 'Student' objects: Alice, Bob, and Cindy, and prints their details.

```
src > Experiment_4_Task_3.cpp > main()
1 #include <iostream>
2 #include <iomanip>
3 #include <string>
4
5 using namespace std;
6
7 class Student
8 {
9     private:
10         string name;
11         string id;
12         double grade;
13
14     public:
15         Student() : grade(0.0) {}
16
17         Student(string n, string i, double g) : name(n), id(i), grade(g)
18         {
19             setName(n);
20             setGrade(g);
21             setId(i);
22         }
23
24         void setName(string n)
25         {
26             if (!n.empty())
27             {
28                 name = n;
29             }
30             else
31             {
```

The output window at the bottom shows the execution results of the program:

```
PS D:\OneDrive\Desktop\Experiment_4> ./bin/Experiment_4_Task_3.exe
Student 1: Name: Alice, ID: 2024001, Grade: 95.00
Student 2: Name: Bob, ID: 2024002, Grade: 82.50
Student 3: Name: Cindy, ID: 2024003, Grade: 76.00
PS D:\OneDrive\Desktop\Experiment_4>
```

On the right side of the interface, there is a sidebar with a 'Messages' tab showing a notification about the 'Plan agent' and a 'Run and Debug' section with a 'Run' button.

Task 4: Bank Account Class (Advanced OOP & Behavior Modeling)

1. **Description:** Describe the class design and program logic in words or with a flow diagram. Do not paste the source code here.

- 1) Create a BankAccount class with private data members accountNumber, ownerName, and balance in the header file BankAccount.h. Declare the functions displayBalance(), deposit(double amount), withdraw(double amount). And add a parameterized constructor and a destructor.

```
class BankAccount
{
    private:
        std::string accountNumber;
        std::string ownerName;
        double balance;

    public:
        BankAccount(std::string accNum, std::string owner, double initialBalance);

        ~BankAccount();

        void displayBalance() const;

        void deposit(double amount);

        void withdraw(double amount);
};
```

- 2) Define the functions in BankAccount.cpp.

```
#include "BankAccount.h"
#include <iostream>
#include <iomanip>

using namespace std;

BankAccount::BankAccount(string accNum, string owner, double initialBalance) :
accountNumber(accNum), ownerName(owner), balance(initialBalance)
{
    if (balance < 0)
    {
        balance = 0;
        cout << "Warning: Initial balance cannot be negative. Setting balance to 0." <<
endl;
    }
}

BankAccount::~BankAccount()
{
    cout << "BankAccount object destroyed." << endl;
}

void BankAccount::displayBalance() const
```

```

{
    cout << fixed << setprecision(2) << "Current balance: " << balance << endl;
}

void BankAccount::deposit(double amount)
{
    if (amount > 0)
    {
        balance += amount;
        cout << fixed << setprecision(2) << "Deposit " << amount << " successful." <<
endl;
    }
    else
    {
        cout << "Deposit failed: amount must be positive." << endl;
    }
}

void BankAccount::withdraw(double amount)
{
    if (amount <= 0)
    {
        cout << "Withdrawal failed: amount must be positive." << endl;
    }
    else if (amount > balance)
    {
        cout << "Withdrawal failed: insufficient balance." << endl;
    }
    else
    {
        balance -= amount;
        cout << fixed << setprecision(2) << "Withdraw " << amount << " successful." <<
endl;
    }
}
}

```

- 3) In main(), create a menu-driven program with four choices: 1. Display balance; 2. Withdraw; 3. Deposit; 4. Exit. For choices 2 and 3, ask the user to enter an amount using switch-case. Repeat the menu until Exit and handle invalid choices.

```

BankAccount account("123456789", "John Doe", 1000.0);

int choice;
double amount;

do
{
    cout << "==== Bank Account Menu =====" << endl;
    cout << "1. Display Balance" << endl;
    cout << "2. Withdraw" << endl;

```

```
cout << "3. Deposit" << endl;
cout << "4. Exit" << endl;
cout << "Enter your choice: ";

cin >> choice;

switch (choice)
{
case 1:
    account.displayBalance();
    break;

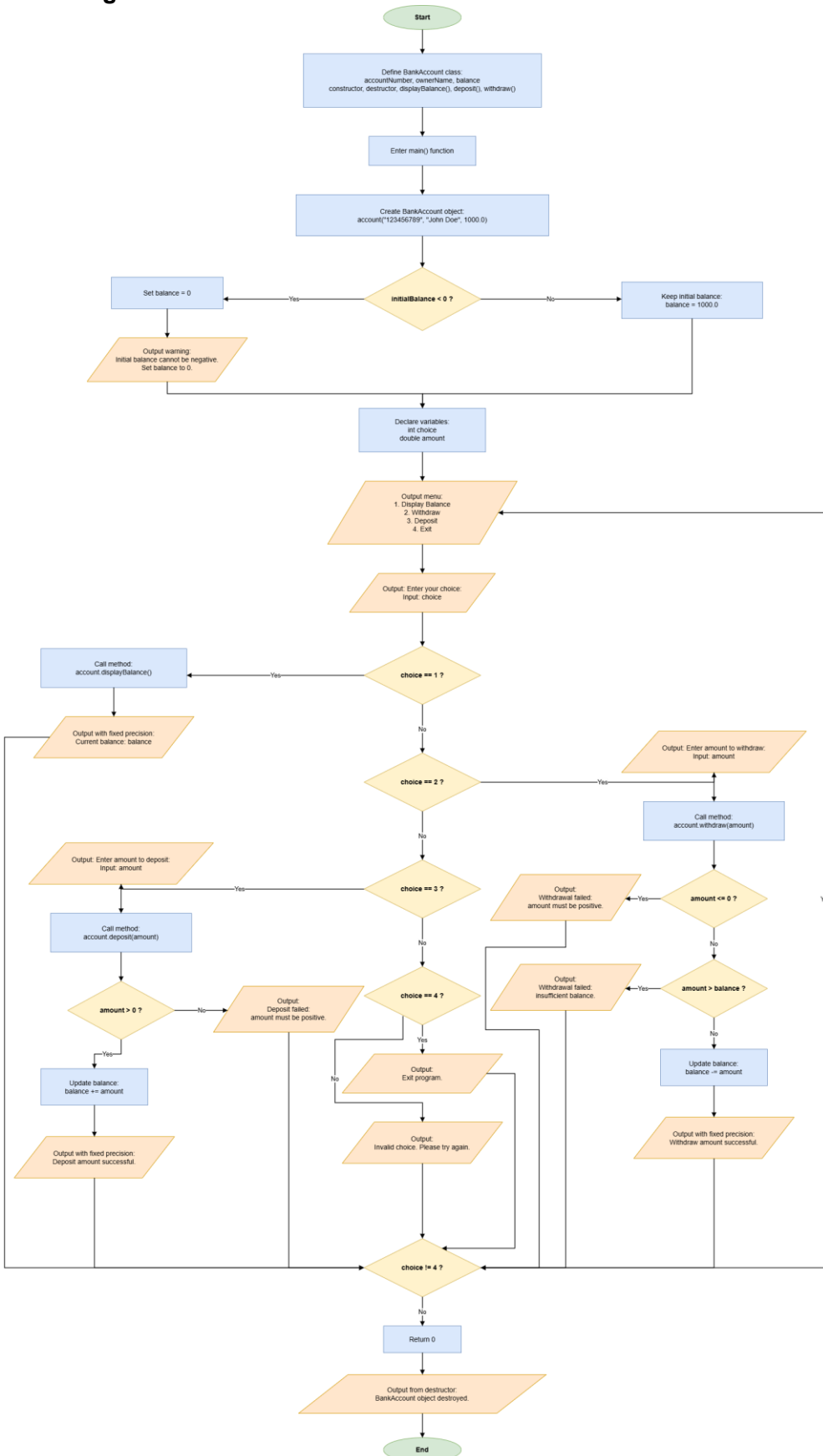
case 2:
    cout << "Enter amount to withdraw: ";
    cin >> amount;
    account.withdraw(amount);
    break;

case 3:
    cout << "Enter amount to deposit: ";
    cin >> amount;
    account.deposit(amount);
    break;

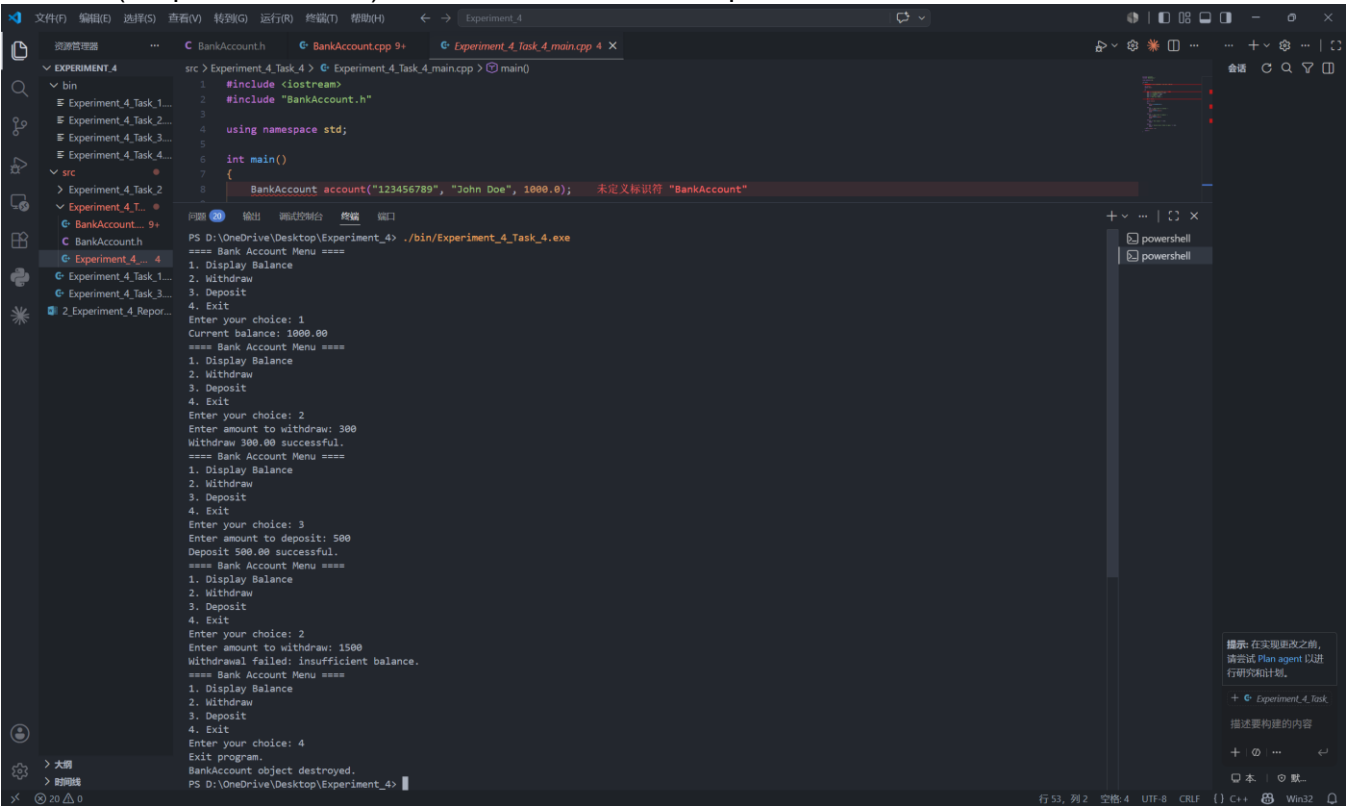
case 4:
    cout << "Exit program." << endl;
    break;

default:
    cout << "Invalid choice. Please try again." << endl;
    break;
}
} while (choice != 4);
```

2. Flow Diagram:



3. Results (Output Screenshot): The screenshot of the output of the code.



```
src > Experiment_4_Task_4 > C: Experiment_4_Task_4_main.cpp > main()
1 #include <iostream>
2 #include "BankAccount.h"
3
4 using namespace std;
5
6 int main()
7 {
8     BankAccount account("123456789", "John Doe", 1000.0); 未定义标识符 "BankAccount"
9 }

PS D:\OneDrive\Desktop\Experiment_4> .\bin\Experiment_4_Task_4.exe
==== Bank Account Menu ====
1. Display Balance
2. Withdraw
3. Deposit
4. Exit
Enter your choice: 1
Current balance: 1000.00
==== Bank Account Menu ====
1. Display Balance
2. Withdraw
3. Deposit
4. Exit
Enter your choice: 2
Enter amount to withdraw: 300
Withdraw 300.00 successful.
==== Bank Account Menu ====
1. Display Balance
2. Withdraw
3. Deposit
4. Exit
Enter your choice: 3
Enter amount to deposit: 500
Deposit 500.00 successful.
==== Bank Account Menu ====
1. Display Balance
2. Withdraw
3. Deposit
4. Exit
Enter your choice: 2
Enter amount to withdraw: 1500
Withdrawal failed: insufficient balance.
==== Bank Account Menu ====
1. Display Balance
2. Withdraw
3. Deposit
4. Exit
Enter your choice: 4
Exit program.
BankAccount object destroyed.
PS D:\OneDrive\Desktop\Experiment_4>
```

IV. Extended Questions

1. In Task 1 and Task 3, why should data members such as width, height, and grade be declared as private instead of public? How do setter functions help protect data validity?

Data members such as width, height, and grade should be declared as private because they represent the internal state of an object. If they are public, outside code can modify them directly without any checking. This may cause invalid object states. For example, a rectangle may have a negative width or height, and a student may have a grade less than 0 or greater than 100. These values are logically incorrect.

Declaring these data members as private follows the principle of encapsulation. The class hides its internal data and only provides public member functions as controlled interfaces. In this way, the object can protect its own data and maintain a valid state.

Setter functions help protect data validity because they check input values before assigning them to private data members. For example, `setWidth()` and `setHeight()` only accept positive values. If the input is invalid, the original valid value will not be overwritten. Similarly, `setGrade()` only accepts values from 0 to 100. If the grade is outside this range, the function rejects it and gives a warning. Therefore, setter functions make the program safer and prevent invalid data from being stored in objects.

2. Read the following code and answer the questions.

```
#include <iostream>
#include <string>
using namespace std;

class Demo {
public:
    Demo(string n) {
        name = n;
        cout << "Constructor: " << name << endl;
    }

    ~Demo() {
        cout << "Destructor: " << name << endl;
    }

private:
    string name;
};

void createObject() {
    cout << "Inside createObject" << endl;
    Demo b("B");
}
```

```

int main() {
    Demo a("A");

    createObject();

    cout << "End of main" << endl;

    return 0;
}

```

1) What is the output of this program?

```

Constructor: A
Inside createObject
Constructor: B
Destructor: B
End of main
Destructor: A

```

2) Why does the program produce this output? Explain the lifetime of object a and object b.

The program produces this output because constructors are called when objects are created, and destructors are called when objects go out of scope.

In main(), object a is created first by the statement `Demo a("A");`. Therefore, the constructor of a is called and the program prints:

```

Constructor: A

```

Then the function `createObject()` is called. When the program enters `createObject()`, it first prints:

```

Inside createObject

```

Inside this function, object b is created by the statement `Demo b("B");`. Therefore, the constructor of b is called and the program prints:

```

Constructor: B

```

Object b is a local object inside `createObject()`. Its lifetime only lasts within this function. When `createObject()` finishes, object b goes out of scope, so its destructor is called immediately. Therefore, the program prints:

```

Destructor: B

```

After `createObject()` returns, the program continues executing `main()` and prints:

```

End of main

```

Object a is a local object in `main()`. Its lifetime lasts until `main()` finishes. After `return 0;` is executed, object a goes out of scope, so its destructor is called at the end. Therefore, the program finally prints:

Destructor: A

In conclusion, object b is destroyed earlier because it belongs to `createObject()`, while object a is destroyed later because it belongs to `main()`.

V. Summary and Reflections

In this experiment, I practiced the basic concepts and programming methods of object-oriented programming in C++. The main contents included class definition, private data members, public member functions, constructors, destructors, encapsulation, arrays of objects, and multi-file program structure.

Through Task 1 and Task 3, I learned how to use classes to describe objects such as rectangles and students. I also understood that private data members can protect object data, while setter functions can check input values before updating them. This helps prevent invalid values such as negative dimensions or grades outside the range of 0 to 100.

Through Task 2, I practiced separating a class into a header file and an implementation file. The Time class also helped me understand how to place program logic inside member functions, especially the overflow handling of seconds, minutes, and hours.

Through Task 4, I learned how to model real object behavior with member functions. The BankAccount class used `deposit`, `withdraw`, and `displayBalance` functions to simulate basic bank account operations. The menu-driven program also strengthened my understanding of loops, switch-case statements, and user interaction.

During the experiment, the main difficulties were handling boundary conditions and compiling multi-file programs correctly. I solved these problems by checking the logic step by step and compiling the related .cpp files together. Overall, this experiment helped me better understand object-oriented programming and improved my ability to design clearer and more modular C++ programs. In the future, I can further improve the programs by adding more complete input validation and making the class design more flexible.